

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск набора подстрок в строке
Вариант 3

Студент гр. 4345

Трушкин А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Ахо-Корасик

Постановка задачи

Часть 1. Разработайте программу, решающую задачу точного поиска набора образцов.

Ввод: Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$). Вторая - число $n (1 \leq n \leq 3000)$, каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$. Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Вывод: Все вхождения образцов из P в T . Каждое вхождение образца в текст представить в виде двух чисел - i и p . Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1). Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Ввод

NTAG

3

TAGT

TAG

T

Вывод

2 2

2 3

Часть 2. Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*. В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T . Например, образец $ab??c?a$ с джокером $??$ встречается дважды в тексте $xabvccbababcsaxxabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T. Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида ??? недопустимы. Все строки содержат символы из алфавита {A,C,G,T,N}

Вход:

Текст (T, $1 \leq |T| \leq 1000000$)

Шаблон (P, $1 \leq |P| \leq 40$)

Символ джокера

Выход: Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер). Номера должны выводиться в порядке возрастания.

Ввод

ACTANCA

A\$ \$A\$

\$

Вывод

1

Вариант 3. Вычислить длину самой длинной цепочки из суффиксных ссылок и самой длинной цепочки из конечных ссылок в автомате.

Выполнение работы

Задание 1

Поиск набора образцов в тексте в данной работе выполняется за один проход с линейной сложностью по времени. Это достигается за счет объединения всех искомых строк в специальный ориентированный граф — префиксное дерево (бор), которое затем преобразуется в конечный автомат.

Построение автомата происходит в два последовательных этапа. На первом этапе (функция `build_trie`) формируется классическое префиксное дерево. Алгоритм считывает каждый шаблон символ за символом, прокладывая новые пути в графе только в случае отсутствия альтернативных переходов. Это исключает дублирование общих начальных фрагментов строк. Вершины, на которых завершается ввод каждого шаблона, помечаются как терминальные с сохранением номеров соответствующих образцов.

На втором этапе (функция `build_automaton`) дерево преобразуется в автомат посредством расчета суффиксных и конечных (сжатых) переходов. Вычисление связей выполняется методом обхода в ширину (BFS), что гарантирует корректность ссылок на каждом уровне и защищает программу от переполнения стека.

- **Суффиксные ссылки (`suff_link`)** связывают вершины так, чтобы при возникновении ошибки (несовпадении символа текста с веткой бора) автомат мог мгновенно переключиться в состояние, представляющее максимальный совпадающий суффикс уже прочитанной части.
- **Конечные (сжатые) ссылки (`exit_link`)** оптимизируют обход терминальных состояний. Если по суффиксным ссылкам выше есть терминальные вершины, конечная ссылка указывает прямо на ближайшую из них, позволяя регистрировать вложенные вхождения за один шаг.

В процессе поиска в тексте (`search_text`) указатель текущего символа в тексте движется строго вперед. Автомат переходит по состояниям согласно входящим буквам, совершая откаты по суффиксным переходам в случае отсутствия прямой дуги. При достижении терминального узла или проходе по цепочке конечных ссылок программа вычисляет стартовые позиции вхождений на основе длин совпавших шаблонов.

Вычисление метрик связности графа (Вариант 3)

Для выполнения требований варианта разработана процедура `calculate_longest_chains`. После завершения построения автомата алгоритм собирает все вершины в общий массив. Для каждого узла вычисляется глубина вложенности:

1. Путем последовательного перехода по ссылкам `suff_link` до корня находится максимальная длина суффиксного пути.
2. Путем аналогичного перехода по ссылкам `exit_link` до пустого указателя (`None`) вычисляется максимальная глубина цепочки терминальных совпадений.

Задание 2

Для реализации поиска с джокерами логика автомата была расширена концепцией декомпозиции. Исходный шаблон разделяется по символу маски на фрагменты, не содержащие джокеров. Для каждого фрагмента фиксируется его длина и относительное смещение относительно начала общего шаблона.

Автомат Ахо-Корасика строится только для этих безмасочных фрагментов. В процессе сканирования текста при обнаружении любого из фрагментов программа рассчитывает, в какой позиции текста должен был начаться весь исходный шаблон, чтобы этот фрагмент совпал на своем месте. В массиве счетчиков `C` (размером с длину текста) инкрементируется ячейка, соответствующая этому стартовому индексу. По завершении обхода те индексы, где значение счетчика совпало с количеством безмасочных фрагментов, признаются валидными вхождениями исходного шаблона с джокерами.

Оценка вычислительной сложности

Базовый поиск (Задание 1):

- Время: $O(N + \sum M_i)$, где N — длина текста, а $\sum M_i$ — объем словаря шаблонов. Построение структуры требует времени, пропорционального размеру шаблонов, а поиск по тексту происходит за один проход. Анализ длин цепочек (Вариант 3)

выполняется за линейное время от количества вершин $O(V)$ благодаря отсутствию циклов в дереве ссылок.

- Память: $O(\sum M_i)$ для хранения дерева переходов и терминальных списков.

Поиск с масками (Задание 2):

- Время: $O(N + \sum L_i + |P|)$, где L_i — длины безмасочных кусков, а $|P|$ — полная длина исходного шаблона. Проход по массиву счетчиков в конце занимает $O(N)$ времени.
- Память: $O(N + \sum L_i)$, обусловлена необходимостью выделения памяти под массив счетчиков S размером с длину текста и дерево переходов для фрагментов.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование task1

№	Входные данные	Выходные данные
1	ACGTACGT 3 CGT AC GT	1 2 2 1 3 3 5 2 6 1 7 3 Самая длинная цепочка суффиксных ссылок: 2 Самая длинная цепочка конечных ссылок: 1
2	ANANA 2 ANA AN	1 1 1 2 3 1 3 2 Самая длинная цепочка суффиксных ссылок: 2 Самая длинная цепочка конечных ссылок: 0

3	CGT 2 AAAA TT	Самая длинная цепочка суффиксных ссылок: 4 Самая длинная цепочка конечных ссылок: 0
---	------------------------	--

Таблица 2 – Тестирование task2

№	Входные данные	Выходные данные
1	AGATAC G?T ?	2 Самая длинная цепочка суффиксных ссылок: 1 Самая длинная цепочка конечных ссылок: 0
2	ANANA A?A ?	1 3 Самая длинная цепочка суффиксных ссылок: 1 Самая длинная цепочка конечных ссылок: 0

Выводы

В ходе лабораторной работы был успешно реализован алгоритм Ахо-Корасика для быстрого поиска нескольких подстрок за один проход по тексту. Использование префиксного дерева и автоматическое построение суффиксных и сжатых ссылок методом обхода в ширину позволили полностью исключить лишние сравнения символов и избежать возвратов по тексту назад.

Также алгоритм был адаптирован для поиска шаблонов с джокерами с помощью разбиения строки на чистые части и подсчета их совпадений, что позволило сохранить высокую скорость работы программы. Дополнительно реализованный анализ структуры автомата обеспечил точное нахождение максимальной длины цепочек суффиксных и конечных ссылок согласно Варианту 3. Программа показала стабильную работу, высокую скорость вычислений и экономное использование памяти на всех этапах тестирования.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл task1.py

```
from collections import deque

class AhoCorasickNode:
    _id_counter = 0

    def __init__(self, char="#"):
        self.id = AhoCorasickNode._id_counter
        AhoCorasickNode._id_counter += 1
        self.char = char
        self.transitions = {}
        self.pattern_indices = []
        self.suff_link = None
        self.exit_link = None

def build_trie(patterns):
    AhoCorasickNode._id_counter = 0
    root = AhoCorasickNode()

    print("--- 1. Построение бора (TRIE) ---")
    for idx, pattern in enumerate(patterns, 1):
        print(f"\nДобавляем шаблон №{idx}: '{pattern}'")
        curr = root
        for char in pattern:
            if char not in curr.transitions:
                new_node = AhoCorasickNode(char)
                curr.transitions[char] = new_node
                print(f"    Создаем новый узел №{new_node.id} по символу '{char}' из узла №{curr.id}")
            else:
                print(f"    Переходим в существующий узел №{curr.transitions[char].id} по символу '{char}'")
                curr = curr.transitions[char]
            curr.pattern_indices.append(idx)
```

```

        print(f"    Узел №{curr.id} становится терминальным для шаблона
№{idx}")
    return root

def build_automaton(root):
    print("\n--- 2. Построение суффиксных и конечных ссылок (BFS) ---")
    queue = deque()

    for char, child in root.transitions.items():
        child.suff_link = root
        child.exit_link = None
        print(f"Узел первого уровня №{child.id} ('{char}'): суф. ссылка
-> Корень №{root.id}")
        queue.append(child)

    while queue:
        curr = queue.popleft()

        for char, child in curr.transitions.items():
            temp = curr.suff_link
            while temp is not None and char not in temp.transitions:
                temp = temp.suff_link

            if temp is None:
                child.suff_link = root
            else:
                child.suff_link = temp.transitions[char]

            suff = child.suff_link
            if suff.pattern_indices:
                child.exit_link = suff
            else:
                child.exit_link = suff.exit_link

        print(f"Узел №{child.id} ('{char}'): "
              f"суф. ссылка -> №{child.suff_link.id}, "
              f"кон. ссылка -> №{child.exit_link.id if child.exit_link
else 'None'}")

```

```

        queue.append(child)

def get_all_nodes(root):
    """Вспомогательная функция для сбора всех вершин автомата"""
    nodes = []
    queue = deque([root])
    while queue:
        curr = queue.popleft()
        nodes.append(curr)
        for child in curr.transitions.values():
            queue.append(child)
    return nodes

def calculate_longest_chains(root):
    """Реализация Варианта 3: вычисление максимальных длин цепочек
    ссылок"""
    nodes = get_all_nodes(root)
    max_suff_chain = 0
    max_exit_chain = 0

    for node in nodes:
        suff_len = 0
        curr = node.suff_link
        while curr is not None and curr != root:
            suff_len += 1
            curr = curr.suff_link
        if curr == root:
            suff_len += 1
        max_suff_chain = max(max_suff_chain, suff_len)

        exit_len = 0
        curr = node.exit_link
        while curr is not None:
            exit_len += 1
            curr = curr.exit_link
        max_exit_chain = max(max_exit_chain, exit_len)

    return max_suff_chain, max_exit_chain

```

```

def search_text(text, root, patterns):
    print("\n--- 3. Процесс поиска в тексте ---")
    results = []
    curr = root

    for i, char in enumerate(text):
        print(f"Символ [{i+1}]: '{char}'. Текущий узел автомата: №{curr.id}")

        while curr is not None and char not in curr.transitions:
            print(f"Нет перехода по '{char}'. Откатываемся: №{curr.id}
-> №{curr.suff_link.id if curr.suff_link else 'None'}")
            curr = curr.suff_link

        if curr is None:
            curr = root
            print(f"Вернулись в корень №{root.id}")
        else:
            curr = curr.transitions[char]
            print(f"Успешный переход в узел №{curr.id} ('{char}'))")

        temp = curr
        while temp is not None:
            for idx in temp.pattern_indices:
                start_pos = i - len(patterns[idx - 1]) + 2
                print(f"Найдено совпадение: шаблон №{idx}
('{patterns[idx-1]}') со стартовой позиции {start_pos}")
                results.append((start_pos, idx))
            temp = temp.exit_link

    return results

def main():
    text = input("Введите текст: ").strip()
    n = int(input("Введите количество шаблонов: ").strip())

    patterns = []

```

```

for i in range(n):
    patterns.append(input(f"Шаблон №{i+1}: ").strip())

root = build_trie(patterns)
build_automaton(root)

matches = search_text(text, root, patterns)

matches.sort(key=lambda x: (x[0], x[1]))

max_suff, max_exit = calculate_longest_chains(root)

print("\n--- Итоговые результаты ---")
print("Вхождения (Позиция Шаблон):")
for pos, pattern_idx in matches:
    print(f"{pos} {pattern_idx}")

print(f"\nСамая длинная цепочка суффиксных ссылок: {max_suff}")
print(f"Самая длинная цепочка конечных ссылок: {max_exit}")

if __name__ == "__main__":
    main()

```

Файл task2.py

```

from collections import deque

class AhoCorasickNode:
    _id_counter = 0

    def __init__(self, char="#"):
        self.id = AhoCorasickNode._id_counter
        AhoCorasickNode._id_counter += 1
        self.char = char
        self.transitions = {}
        self.parts_info = []
        self.suff_link = None
        self.exit_link = None

```

```

def build_trie(parts):
    AhoCorasickNode._id_counter = 0
    root = AhoCorasickNode()

    print("--- 1. Построение бора (TRIE) ---")
    for idx, (pattern, offset) in enumerate(parts, 1):
        print(f"\nДобавляем подстроку №{idx}: '{pattern}' (смещение в шаблоне: {offset})")
        curr = root
        for char in pattern:
            if char not in curr.transitions:
                new_node = AhoCorasickNode(char)
                curr.transitions[char] = new_node
                print(f"    Создаем новый узел №{new_node.id} по символу '{char}' из узла №{curr.id}")
            else:
                print(f"    Переходим в существующий узел №{curr.transitions[char].id} по символу '{char}'")
                curr = curr.transitions[char]
            curr.parts_info.append((offset, len(pattern)))
        print(f"    Узел №{curr.id} становится терминальным для подстроки '{pattern}'")
    return root

def build_automaton(root):
    print("\n--- 2. Построение суффиксных и конечных ссылок (BFS) ---")
    queue = deque()

    for char, child in root.transitions.items():
        child.suff_link = root
        child.exit_link = None
        print(f"Узел первого уровня №{child.id} ('{char}'): суф. ссылка -> Корень №{root.id}")
        queue.append(child)

    while queue:
        curr = queue.popleft()
        for char, child in curr.transitions.items():

```

```

        temp = curr.suff_link
        while temp is not None and char not in temp.transitions:
            temp = temp.suff_link

        if temp is None:
            child.suff_link = root
        else:
            child.suff_link = temp.transitions[char]

        suff = child.suff_link
        if suff.parts_info:
            child.exit_link = suff
        else:
            child.exit_link = suff.exit_link

        print(f"Узел №{child.id} ('{char}'): "
              f"суф. ссылка -> №{child.suff_link.id}, "
              f"кон. ссылка -> №{child.exit_link.id if child.exit_link
else 'None'}")
        queue.append(child)

def get_all_nodes(root):
    """Сбор всех вершин автомата для анализа структуры"""
    nodes = []
    queue = deque([root])
    while queue:
        curr = queue.popleft()
        nodes.append(curr)
        for child in curr.transitions.values():
            queue.append(child)
    return nodes

def calculate_longest_chains(root):
    """Реализация Варианта 3: вычисление максимальных длин цепочек
ссылок"""
    nodes = get_all_nodes(root)
    max_suff_chain = 0
    max_exit_chain = 0

```



```

for node in nodes:
    suff_len = 0
    curr = node.suff_link
    while curr is not None and curr != root:
        suff_len += 1
        curr = curr.suff_link
    if curr == root:
        suff_len += 1
    max_suff_chain = max(max_suff_chain, suff_len)

    exit_len = 0
    curr = node.exit_link
    while curr is not None:
        exit_len += 1
        curr = curr.exit_link
    max_exit_chain = max(max_exit_chain, exit_len)

return max_suff_chain, max_exit_chain

def search_text(text, root, pattern_len, C):
    print("\n--- 3. Процесс поиска в тексте ---")
    n = len(text)
    curr = root
    for i in range(n):
        char = text[i]
        print(f"Символ [{i+1}]: '{char}'. Текущий узел автомата: №{curr.id}")

        while curr is not None and char not in curr.transitions:
            print(f"Нет перехода по '{char}'. Откатываемся: №{curr.id}
-> №{curr.suff_link.id if curr.suff_link else 'None'}")
            curr = curr.suff_link

        if curr is None:
            curr = root
            print(f"Вернулись в корень №{root.id}")
        else:

```

```

curr = curr.transitions[char]
print(f"    Успешный переход в узел №{curr.id} ('{char}'))

temp = curr
while temp is not None:
    for offset, length in temp.parts_info:
        start_pos = i - offset - length + 1
        if 0 <= start_pos <= n - pattern_len:
            C[start_pos] += 1
            print(f"    -> Найдено совпадение подстроки! Смещение
в шаблоне: {offset}, Длина: {length}")
            print(f"        Увеличиваем счетчик совпадений для
стартовой позиции {start_pos + 1} (C[{start_pos + 1}] = {C[start_pos]})")
            temp = temp.exit_link

def main():
    text = input("Введите текст: ").strip()
    pattern = input("Введите шаблон с джокерами: ").strip()
    joker = input("Введите символ джокера: ").strip()

    parts = []
    current_part = []
    start_idx = -1

    print("\n--- Подготовка шаблона ---")
    for idx, char in enumerate(pattern):
        if char != joker:
            if start_idx == -1:
                start_idx = idx
            current_part.append(char)
        else:
            if current_part:
                parts.append(("".join(current_part), start_idx))
                print(f"Выделена чистая подстрока:
'{''.join(current_part)}', смещение: {start_idx}")
                current_part = []
                start_idx = -1
            if current_part:

```

```

        parts.append(''.join(current_part), start_idx))
        print(f"Выделена чистая подстрока: '{''.join(current_part)}',
смещение: {start_idx}")

    if not parts:
        print("Ошибка: шаблон состоит только из джокеров.")
        return

    C = [0] * len(text)

    root = build_trie(parts)
    build_automaton(root)

    search_text(text, root, len(pattern), C)

    max_suff, max_exit = calculate_longest_chains(root)

    print("\n--- Итоговые результаты ---")
    print("Вхождения полного шаблона (Стартовые позиции):")
    for idx in range(len(text) - len(pattern) + 1):
        if C[idx] == len(parts):
            print(idx + 1)

    print(f"\n Самая длинная цепочка суффиксных ссылок: {max_suff}")
    print(f" Самая длинная цепочка конечных ссылок: {max_exit}")

if __name__ == "__main__":
    main()

```